

IAM overview

This page describes how Google Cloud's Identity and Access Management (IAM) system works and how you can use it to manage access in Google Cloud.

IAM is a tool to manage fine-grained authorization for Google Cloud. In other words, it lets you control *who* can do *what* on *which* resources.

Access in Google Cloud

Every action in Google Cloud requires certain permissions. When someone tries to perform an action in Google Cloud—for example, create a VM instance or view a dataset—IAM first checks if they have the required permissions. If they don't, then IAM prevents them from performing the action.

Giving someone permissions in IAM involves the following three components:

- **Principal:** The identity of the person or system that you want to give permissions to.
- **Role:** The collection of permissions that you want to give the principal.
- **Resource:** The Google Cloud resource that you want to let the principal access.

To give the principal permission to access the resource, you grant them a role on the resource. You grant these roles using an *allow policy*.

Allow policies are directly attached to some Google Cloud resources, which are organized hierarchically (#policy-inheritance)—for example, projects contain service-specific resources. This means that you can grant access to a single resource or a container of resources.

The following sections describe these concepts in more detail.

Principals

In Google Cloud you control access for *principals*. Principals represent one or more identities that have authenticated to Google Cloud.

In the past, principals were referred to as *members*. Some APIs still use that term.

There are various types of principals in IAM, but they can be divided into two broad categories:

- **Human users:** Some IAM principal types represent human users. You use these principal types for managing your employees' access to Google Cloud resources.

Principal types that represent human users include Google Accounts, Google groups, and federated identities in workforce identity pools.

- **Workloads:** Some IAM principal types represent workloads. You use these principal types when managing your workloads' access to Google Cloud resources.

Principal types that represent workloads include service accounts and federated identities in a workload identity pool.

For more information about principals, see [IAM principals \(/iam/docs/principals-overview\)](/iam/docs/principals-overview).

Permissions and roles

Permissions determine what operations are allowed on a resource. In IAM, permissions are typically represented in the form *service.resource.verb*. Often, permissions correspond one-to-one with REST API methods—for example, the `resourcemanager.projects.list` permission lets you list Resource Manager projects.

You can't directly grant permissions to a principal. Instead, you give principals permissions by granting them *roles*.

Roles are collections of permissions. When you grant a role to a principal, you give that principal all of the permissions in that role.

There are three types of roles:

- **Predefined roles:** Roles that are managed by Google Cloud services. These roles contain the permissions needed to perform common tasks for each given service. For example, the Pub/Sub Publisher role (`roles/pubsub.publisher`) provides access to publish messages to a Pub/Sub topic.
- **Custom roles:** Roles that you create that contain only the permissions that you specify. You have complete control over the permissions in these roles. However, they have a higher maintenance burden than predefined roles and there's a limit to the number of custom roles that you can have in your project and in your organization.
- **Basic roles:** Highly permissive roles that provide broad access to Google Cloud services. These roles can be useful for testing purposes, but shouldn't be used in production environments.

For more information about roles and permissions, see [Roles and permissions \(/iam/docs/roles-overview\)](/iam/docs/roles-overview).

Resources

Most Google Cloud services have their own resources. For example, Compute Engine has resources like instances, disks, and subnetworks.

In IAM, you grant roles on a resource. Granting a principal a role on a resource means that the principal can use the permissions in that role to access the resource.

You can grant roles on a subset of Google Cloud resources. For a full list of resources that you can grant roles on, see [Resource types that accept allow policies \(/iam/docs/resource-types-with-policies\)](/iam/docs/resource-types-with-policies).

Google Cloud also has container resources, including projects, folders, and organizations. These container resources are organized hierarchically, which lets child resources inherit the policies of their parent resources. This

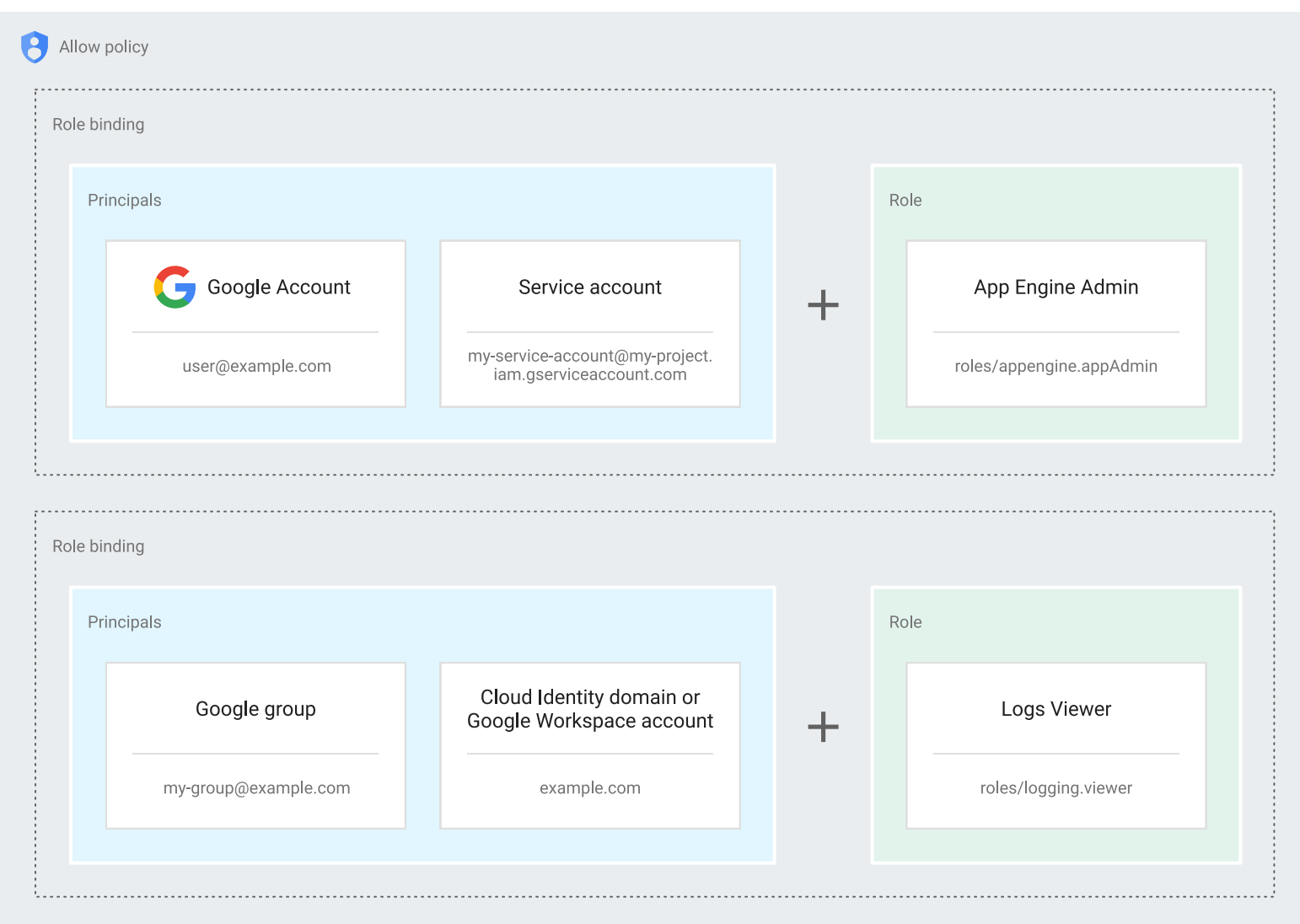
means that granting a principal a role on a container resource gives the principal access to both the container resource *and* the resources in that container. This feature lets you use a single role grant to manage access to multiple resources, including resources that you can't grant roles on directly. For more information, see [Policy inheritance](#) (#policy-inheritance) on this page.

Allow policies

You grant roles to principals using *allow policies*. In the past, these policies were referred to as *IAM policies*.

An allow policy is a YAML or JSON object that's attached to a Google Cloud resource.

The following diagram shows how an allow policy is structured:



Each allow policy contains a list of *role bindings* that associate IAM roles with the principals who are granted those roles.

When an authenticated principal attempts to access a resource, IAM checks the resource's allow policy to determine whether the principal has the required permissions. If the principal is in a role binding that includes a role with the required permissions, then they're allowed to access the resource.

To see examples of allow policies and learn about their structure, see [Understanding allow policies](#) (/iam/docs/allow-policies).

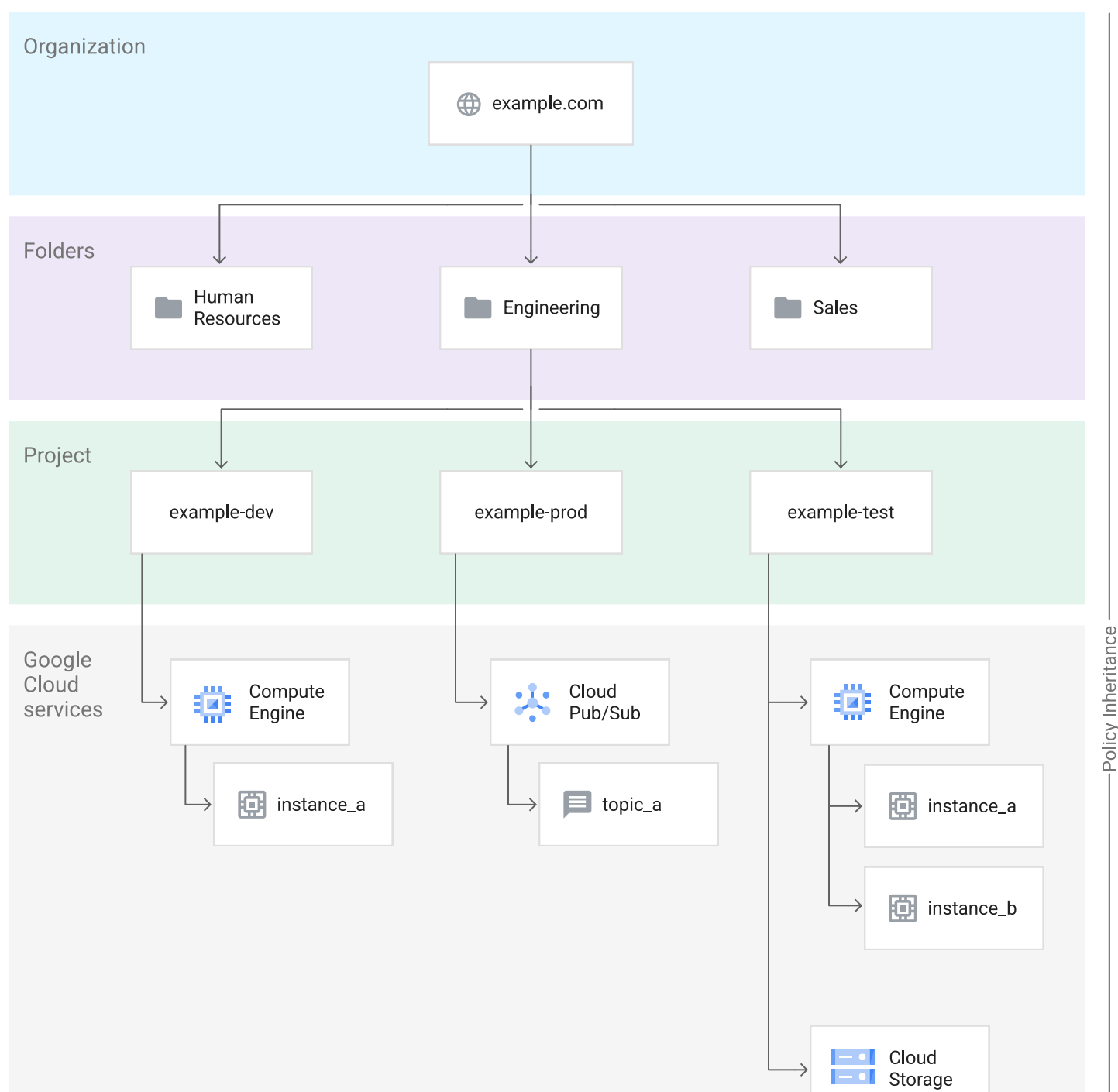
Policy inheritance

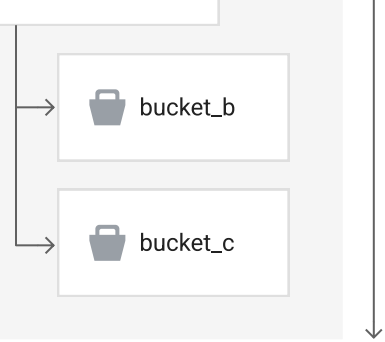
Google Cloud has container resources—such as projects, folders, and organizations—that let you organize your resources in a parent-child hierarchy. This hierarchy is called the *resource hierarchy*.

The Google Cloud resource hierarchy has the following structure:

- The *organization* is the root node in the hierarchy.
- *Folders* are children of the organization, or of another folder.
- *Projects* are children of the organization, or of a folder.
- *Resources* for each service are descendants of projects.

The following diagram is an example of a Google Cloud resource hierarchy:





If you set an allow policy on a container resource, then the allow policy also applies to all resources in that container. This concept is called *policy inheritance*, because descendant resources effectively inherit their ancestor resources' allow policies.

Policy inheritance has the following implications:

- **You can use a single role binding to grant access to multiple resources.** If you want to give a principal access to all resources in a container, then grant them a role on the container instead of on the resources in the container.

For example, if you want to let your security administrator manage allow policies for all resources in your organization, then you could grant them the Security Admin role (`roles/iam.securityAdmin`) on the organization.

- **You can grant access to resources that don't have their own allow policies.** Not all resources accept allow policies, but all resources inherit allow policies from their ancestors. To give a principal access to a resource that can't have its own allow policy, grant them a role on one of the resource's ancestors.

For example, imagine you want to give someone permission to write logs to a log bucket. Log buckets don't have their own allow policies, so to give someone this permission, you can instead grant them the Logs Bucket Writer role (`roles/logging.bucketWriter`) on the project that contains the log bucket.

- **To understand who can access a resource, you need to also view *all* of the allow policies that affect the resource.** To get a complete list of the principals that have access to the resource, you need to view the resource's allow policy *and* the resource's ancestors' allow policies. The union of all of these policies is called the *effective allow policy*.

For more information about policy inheritance for allow policies, see [Using resource hierarchy for access control](https://iam/docs/resource-hierarchy-access-control) (`/iam/docs/resource-hierarchy-access-control`).

Advanced access control

In addition to allow policies, IAM provides the following access control mechanisms to help you refine who has access to what resources:

- **Additional policy types:** IAM offers the following policy types in addition to allow policies:

- **Deny policies:** Deny policies prevent principals from using certain permissions, even if they're granted a role with the permission.
- **Principal access boundary (PAB) policies:** Principal access boundary policies define and enforce the resources a principal is eligible to access. Principals can't access resources that they're not eligible to access, even if they've been granted a role on the resource.

To learn more about these policies, see [Policy types](/iam/docs/policy-types) (/iam/docs/policy-types).

- **IAM Conditions:** IAM Conditions lets you define and enforce conditional, attribute-based access control. You can use conditions in various policy types. For example, you can add a condition to a role binding in an allow policy to ensure that the role is only granted if the condition is met.

You can write conditions based on attributes like the resource in the request and the time of the request.

To learn more about IAM Conditions, see [Overview of IAM Conditions](/iam/docs/conditions-overview) (/iam/docs/conditions-overview).

- **Privileged Access Manager (PAM):** With Privileged Access Manager, you can let principals request and be given temporary, auditable access to resources. For example, you could require that principals request access each time they want to view a sensitive resource instead of permanently granting them a IAM role.

You can also configure whether principals are required to provide justifications or get approvals when they request access.

To learn more about Privileged Access Manager, see [Privileged Access Manager overview](/iam/docs/pam-overview) (/iam/docs/pam-overview).

Consistency model for the IAM API

The [IAM API](/iam/docs/reference/rest) (/iam/docs/reference/rest) is [eventually consistent](https://wikipedia.org/wiki/Eventual_consistency) (https://wikipedia.org/wiki/Eventual_consistency). In other words, if you write data with the IAM API, then immediately read that data, the read operation might return an older version of the data. The changes that you make might also take time to affect access checks.

This consistency model affects how the IAM API works. For example, if you create a service account, then immediately refer to that service account in another request, the IAM API might say that the service account couldn't be found. This behavior occurs because operations are eventually consistent; it can take time for the new service account to become visible to read requests.

What's next

- To learn how to configure identities for Google Cloud, see [Identity management for Google Cloud](/iam/docs/google-identities) (/iam/docs/google-identities).
- To learn how to grant, change, and revoke IAM roles to principals, see [Manage access to projects, folders, and organizations](/iam/docs/granting-changing-revoking-access) (/iam/docs/granting-changing-revoking-access).

- To see available IAM roles, see [Predefined roles \(/iam/docs/roles-permissions\)](/iam/docs/roles-permissions).
- To get help with choosing the most appropriate predefined roles, read [Find the right predefined roles \(/iam/docs/choose-predefined-roles\)](/iam/docs/choose-predefined-roles).
- To see the policy types available in IAM, see [Policy types \(/iam/docs/policy-types\)](/iam/docs/policy-types).

Except as otherwise noted, the content of this page is licensed under the [Creative Commons Attribution 4.0 License \(https://creativecommons.org/licenses/by/4.0/\)](https://creativecommons.org/licenses/by/4.0/), and code samples are licensed under the [Apache 2.0 License \(https://www.apache.org/licenses/LICENSE-2.0\)](https://www.apache.org/licenses/LICENSE-2.0). For details, see the [Google Developers Site Policies \(https://developers.google.com/site-policies\)](https://developers.google.com/site-policies). Java is a registered trademark of Oracle and/or its affiliates.

Last updated 2026-05-08 UTC.